

Date _____

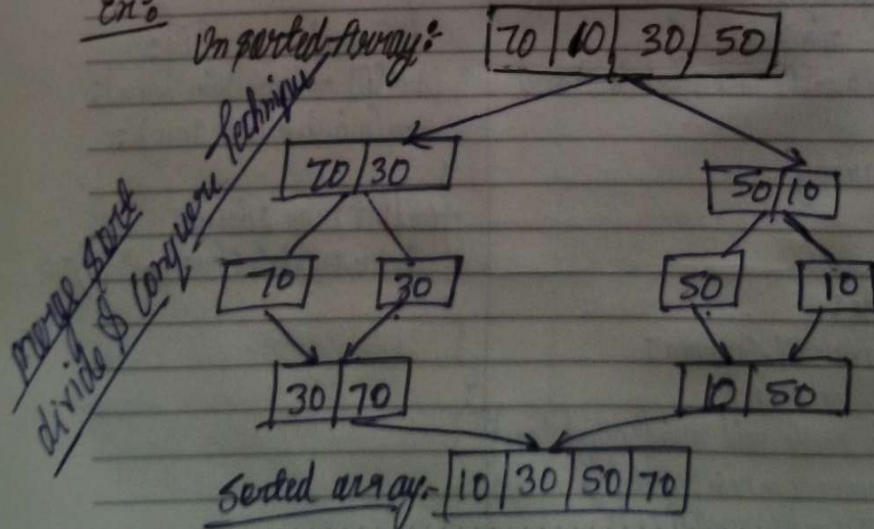
A) Merge Sort :- Week-3

- 1) Problem Statement :- A university's Training and Placement cell has collected the salary packages (in LPA $\times 100$) offered by different companies. Since the package values were entered in random order, they need to be sorted in ascending order before generating placement statistics.
 $\Rightarrow$ Implement the merge sort algorithm to sort the package value.

* Logic :- Merge sort uses the Divide and Conquer technique.

- i) Divide the array into two halves.
- ii) Continue dividing until each subarray contains only one element.
- iii) Merge the smaller sorted arrays into larger sorted arrays.
- iv) While merging, compare elements from both halves and place the smaller element first.
- v) Continue until the complete array becomes sorted.

Ex:



Date _____

2013

* Pseudo Code :-

START

Input size N

Input array

Call merge sort (array, 0, N-1)

Print sorted array

FUNCTION merge sort (array, left, right)

IF $\text{left} < \text{right}$

mid = $(\text{left} + \text{right}) / 2$

merge sort (left half)

merge sort (right half)

merge (left, mid, right)

END FUNCTION

FUNCTION merge (left, mid, right)

Create temporary left array

Create temporary right array

Copy elements

Compare both arrays

Store smaller element

Copy remaining element

END FUNCTION

STOP



* Java Code :-

```

import java.util.Scanner;
public class MergeSort {
    static void merge (int arr[], int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int L[] = new int[n1];
        int R[] = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];

        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j])
                arr[k] = L[i];
            else
                arr[k] = R[j];
            i++;
            j++;
            k++;
        }
        while (i < n1) {
            arr[k] = L[i];
            i++;
            k++;
        }
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
    }
}

```

Date

2013

```
static void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = (left + right) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    int arr[] = new int[n];  
  
    for (int i = 0; i < n; i++)  
        arr[i] = sc.nextInt();  
  
    mergeSort(arr, 0, n - 1);  
  
    for (int num : arr)  
        System.out.print(num + " ");  
  
    sc.close();  
}
```



Output

```
6  
115  
334  
553  
412  
668  
1200  
115 334 412 553 668 1200  
=== Code Execution Successful ===
```


- 2) Problem Statement: An e-commerce company records the processing time (in min) of customer orders. To analyze operational efficiency, the processing times must first be sorted using merge sort. After sorting, the program should:
- display the sorted processing times.
 - Find the median processing time.
 - Count the number of orders whose processing time is greater than the median.
 - display the diff. b/w the fastest and slowest processing time

* Logic :-

- Read the no. of orders & processing times.
- Sort the array using merge sort.
- Print the sorted array.
- Find the median:
 - if "N" is odd, the median is the middle element.
 - if "N" is even, the " " " " avg. of the two mid elements.
- Count how many values are greater than the median.
- Calculate the diff. b/w the max. and min. values.
- display all the required results.

Date _____

2013

* Pseudo code :-

Start

Read N

Read array

Call mergeSort()

Print sorted array

IF N is odd

 median = mid element

ELSE

 median = avg. of two mid elements

count = 0

FOR each element

 IF element > median

 count++

diff. = last element - first element

Print median

Print count

Print difference

Stop

Date _____

2013

* Java Code :-

```
import java.util.Scanner;
public class MergeSort2 {
    static void merge (int arr[], int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int L[] = new int [n1];
        int R[] = new int [n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];

        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];
        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j])
                arr[k++] = L[i++];
            else
                arr[k++] = R[j++];
        }

        while (i < n1)
            arr[k++] = L[i++];

        while (j < n2)
            arr[k++] = R[j++];
    }
}
```


Date

2013

```

static void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        mergeSort(arr, left, mid, right);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();
    mergeSort(arr, 0, n - 1);
    System.out.println("Sorted Processing Times: ");
    for (int num : arr)
        System.out.print(num + " ");
    System.out.println();
    double median;
    if (n % 2 == 0)
        median = (arr[n / 2] + arr[(n / 2) - 1]) / 2.0;
    else
        median = arr[n / 2];
    int count = 0;
    for (int num : arr)
        if (num > median)
            count++;
    int diff = arr[n - 1] - arr[0];
    System.out.println("Median: " + median);
    System.out.println("Orders Above Median: " + count);
    System.out.println("Difference: " + diff);
    sc.close();
}

```

Output

```

7
12
25
18
30
15
22
40
Sorted Processing Times:
12 15 18 22 25 30 40
Median: 22.0
Orders Above Median: 3
Difference: 28

```


Date _____

2013

B) Quick Sort:

1) Problem Statement :- A cloud infrastructure company collects the response time (in millise.) of its servers. Since the response times are stored in random order, they must be sorted in ascending order using Quick Sort. Built-in sorting methods are not allowed.

* Logic :- Quick sort also follows the divide and conquer approach.

- i) Choose the last element as the pivot.
- ii) Place the pivot in its correct position.
- iii) Move all elements smaller than the pivot to the left.
- iv) Move all larger element to the right.
- v) Repeat the process recursively for the left & right subarrays.
- vi) Continue until the array is completely sorted.

* Pseudo code :

Start

Read N

Read array

Call quicksort (array, 0, N-1)

Print array

Function quicksort (low, high)

If low < high

 pivotIndex = partition (low, high)

 quicksort (low, pivotIndex - 1)

 quicksort (pivotIndex + 1, high)

End Function

Function partition (low, high)

 pivot = last element

 i = low - 1

 For j = low to high - 1

 If array[j] < pivot

 i++

 swap array[i] and array[j]

 swap pivot with array[i+1]

 Return i+1

END function

STOP.



Date

2013

*Java Code:-

```

import java.util.Scanner;
public class QuickSortEasy {
    static int partition(int arr[], int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i+1];
        arr[i+1] = arr[high];
        arr[high] = temp;
        return i+1;
    }

    static void quicksort(int arr[], int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quicksort(arr, low, pi-1);
            quicksort(arr, pi+1, high);
        }
    }

    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();
        int arr[] = new int [n];
        for (int i=0; i<n; i++)
            arr[i] = sc.nextInt();
        quicksort(arr, 0, n-1);
        for (int num: arr)
            System.out.print(num + " ");
    }
}

```

Output

6

120

45

78

200

60

95

45 60 78 95 120 200

=== Code Execution Successful ===

Date _____

(2) Problem Statement: A financial analytics company receives trade values from different stock exchanges. Before analysis, the values must be sorted in descending order using Quicksort.

After sorting:

- (i) display the sorted values.
- (ii) display the TOP 5 values.
- (iii) find the avg. of the TOP 5 values.
- (iv) Count how many values are greater than the overall avg. of all trade values.

* Logic:- (i) Read the no. of trade values.

(ii) Read all values.

(iii) Use Quicksort in descending order.

(iv) print the sorted array.

(v) print the first five elements.

(vi) calculate the avg. of the TOP 5 values.

(vii) calculate the avg. of all values.

(viii) Count how many values are greater than the overall avg.

(ix) print the results.

* Pseudo code:-

Start

Read N

Read array

Call quicksort Descending()

print sorted array

sumTop = first five values

avg. Top = sumTop/5

overall Avg. = total sum/N

count = 0

FOR each values

IF value > overall Avg.

count++

Print Top 5

Print avg. Top

Print count

STOP.

* Java Code :-

```
import java.util.Scanner;
public class QuickSort2 {
    static int partition(int arr[], int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i+1];
        arr[i+1] = arr[high];
        arr[high] = temp;
        return i+1;
    }
    static void quickSort(int arr[], int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi-1);
            quickSort(arr, pi+1, high);
        }
    }
    public static void main (String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        long total = 0;
```


2013

Date

```

for (int i=0; i<n; i++) {
    arr[i] = sc.nextInt();
    total += arr[i];
}
quickSort(arr, 0, n-1);
System.out.println("Sorted Trade Values:");
for (int x: arr)
    System.out.println(x + " ");
System.out.println();
System.out.print("Top 5: ");
long topSum = 0;
for (int i=0; i<5; i++) {
    System.out.print(arr[i] + " ");
    topSum += arr[i];
}
double topAvg = topSum / 5.0;
double overallAvg = total / (double)n;
int count = 0;
for (int value: arr)
    if (value > overallAvg)
        count++;
System.out.println();
System.out.println("Avg. of Top 5: " + topAvg);
System.out.println("Values Above Overall Avg: " + count);
}
}

```



Output

```

8
500
1200
700
2000
900
1500
3000
2500
Sorted Trade Values:
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200

```

(C) Heap Sort :-

1) Problem Statement :- An online gaming platform stores player scores in random order after a tournament. Before generating the leader-board, the scores must be sorted in ascending order using Heap Sort.

* Logic :- Heap Sort uses a Binary Heap data structure.

- (i) Convert the array into a Max Heap.
- (ii) The largest element will be at the root.
- (iii) Swap the root with the last element.
- (iv) Reduce the heap size by one.
- (v) Restore the Max Heap using Heapify.
- (vi) Repeat until all elements are sorted.

* Pseudocode :-

Start
 Read N
 Read array
 Build Max Heap
 For $i = N-1$ to 1
 swap root with arr[i]
 Heapify (root)
 print sorted array
 Stop

IF right exists AND right > largest
 largest = right

IF largest $\neq$ root
 swap (root, largest)
 Heapify (largest)

END Function.

Function Heapify (array, size, root)

 largest = root

 left = $2 * \text{root} + 1$

 right = $2 * \text{root} + 2$

 IF left exists AND left > largest
 largest = left

Date _____

2013

Java Code:-

```
import java.util.Scanner;
```

```
public class HeapSort1 {
```

```
    static void heapify(int arr[], int n, int i) {
```

```
        int largest = i;
```

```
        int left = 2 * i + 1;
```

```
        int right = 2 * i + 2;
```

```
        if (left < n && arr[left] > arr[largest])  
            largest = left;
```

```
        if (right < n && arr[right] > arr[largest])  
            largest = right;
```

```
        if (largest != i) {
```

```
            int temp = arr[i];
```

```
            arr[i] = arr[largest];
```

```
            arr[largest] = temp;
```

```
            heapify(arr, n, largest);
```

```
        }
```

```
    }
```

```
    static void heapSort(int arr[]) {
```

```
        int n = arr.length;
```

```
        for (int i = n/2 - 1; i >= 0; i--) {
```

```
            heapify(arr, n, i);
```

```
        }  
        for (int i = n - 1; i > 0; i--) {
```

```
            int temp = arr[0];
```

```
            arr[0] = arr[i];
```

```
            arr[i] = temp;
```

```
            heapify(arr, i, 0);
```

```
        }
```

```
    }
```



2013

Date _____

```
public static void main (String[] args) {  
    Scanner sc = new Scanner (System.in);  
    int n = sc.nextInt();  
    int arr[] = new int[n];
```

```
    for (int i=0; i<n; i++)  
        arr[i] = sc.nextInt();
```

```
    heapSort(arr);
```

```
    for (int x: arr)  
        System.out.print(x + " ");
```

```
    }  
}
```

Output

6

850

1200

950

600

1500

1000

600 850 950 1000 1200 1500

=== Code Execution Successful ===

Date _____

2013

2) Problem Statement:- A hospital records emergency response times for patients. The response times must first be sorted using Heap Sort.

After sorting:

- (i) display sorted response times.
- (ii) find the fastest response time.
- (iii) find the slowest response time.
- (iv) Calculate the avg. response time.
- (v) Count the no. of cases faster than the avg.
- (vi) Calculate the percentage of cases faster than the avg.

* Logic:-

- (i) Read all response times.

- (ii) Sort the array using Heap Sort.

- (iii) print the sorted array.

- (iv) Fastest = first element.

- (v) Slowest = last element.

- (vi) compute avg.

- (vii) count elements smaller than avg.

- (viii) calculate percentage.

* Pseudocode:-

Start

Read N

Read array

Heap Sort

Print sorted array

fastest = first element

slowest = last element

sum = 0

FOR each value

 sum += value

avg. = sum / N

count = 0

FOR each value

 IF value < avg.

 count++

percentage = (count * 100) / N

Print all results

STOP



Date _____

* Java Code :-

```
import java.util.Scanner;
public class Heapsort2 {
    static void heapify(int arr[], int n, int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
```

```
        if (left < n && arr[left] > arr[largest])
            largest = left;
```

```
        if (right < n && arr[right] > arr[largest])
            largest = right;
```

```
        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
            heapify(arr, n, largest);
        }
```

```
    }
    static void heapsort(int arr[]) {
        int n = arr.length;
        for (int i = n/2 - 1; i >= 0; i--)
            heapify(arr, n, i);
```

```
        for (int i = n - 1; i > 0; i--) {
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;
            heapify(arr, i, 0);
        }
```

}



Date

```

public static void main (String[] args) {
    Scanner sc = new Scanner (System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    double sum = 0;
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
        sum += arr[i];
    }
    heapSort(arr);
    System.out.println("Sorted Response Times:");
    for (int x: arr)
        System.out.print(x + " ");
    System.out.println();
    int fastest = arr[0];
    int slowest = arr[n-1];
    double avg = sum/n;
    int count = 0;
    for (int value: arr)
        if (value < avg)
            count++;
    double percentage = (count * 100.0) / n;
    System.out.println("Fastest: " + fastest);
    System.out.println("Slowest: " + slowest);
    System.out.println("Avg: " + avg);
    System.out.println("Cases faster than Avg: " + count);
    System.out.println("Percentage: %2f%%", percentage);
}

```

Output

8

12

7

15

9

20

5

10

8

Sorted Response Times:

5 7 8 9 10 12 15 20

Fastest: 5